

Rapport de gestion de projet

Passage du POC au MVP — Chatbot intelligent Puls-Events

Alternance Data Engineer — Puls-Events

Introduction

Puls-Events est une plateforme qui permet à ses utilisateurs de découvrir des événements culturels en fonction de leurs préférences (lieu, période, thématique). Un premier prototype (POC, Projet 11) a validé la faisabilité d'un chatbot capable de répondre à des questions en langage naturel sur les événements de Seine-Saint-Denis (93), grâce à une architecture RAG (Retrieval-Augmented Generation) : recherche sémantique dans une base de 3 880 événements, puis génération de la réponse par un modèle de langage (Mistral). L'objectif de ce projet est de faire évoluer ce prototype vers un MVP (Minimum Viable Product) robuste et scalable, utilisable en conditions réelles. Ce document présente l'analyse des besoins, l'architecture technique retenue, le backlog fonctionnel, le planning et le budget nécessaires à cette transformation.

Analyse et synthèse des besoins formulés par l'équipe

Synthèse du contexte, objectifs métiers et contraintes techniques

Le POC a démontré la faisabilité technique du chatbot, mais son évaluation avec RAGAS a mis en évidence des marges de progression claires :

- Faithfulness (fidélité des réponses au contexte récupéré) : 0,575
- Context Precision (pertinence des informations récupérées) : 0,422
- Context Recall (capacité à retrouver toutes les informations nécessaires) : 0,233 — le point le plus faible

À partir de ces résultats et des attentes exprimées par les équipes produit et marketing, quatre besoins prioritaires ont été formulés pour le MVP :

- Mémoire conversationnelle — comprendre les questions de suivi dans une même conversation
- Contexte géographique — adapter les résultats à la localisation de l'utilisateur ; répond directement à la faiblesse du Recall observée sur le POC
- Recherche web en temps réel — compléter les données internes lorsqu'une information récente ou manquante est demandée
- Monitoring — suivre en continu la performance et la qualité des réponses, ce qui manquait totalement sur le POC

Analyse des utilisateurs cibles et des cas d'usage

Utilisateurs cibles : les visiteurs de la plateforme Puls-Events souhaitant découvrir des événements culturels sans naviguer dans des filtres classiques.

Cas d'usage principaux :

- Rechercher des événements près de chez soi (« Que se passe-t-il ce week-end à Bondy ? »)
- Poser plusieurs questions à la suite sans tout reformuler (« Et à Aubervilliers ? »)
- Vérifier une information récente si la base interne est incomplète

Plan de projet

Jalons :

- Cadrage validé — fin semaine 1
- Architecture validée — fin semaine 3
- Backlog et coûts validés — fin semaine 4
- MVP structuré (mémoire + géographie) — fin semaine 6
- Recherche web et monitoring spécifiés — fin semaine 8
- Validation finale — fin semaine 10

Échéancier :

Semaine	Contenu
S1	Cadrage du besoin
S2	Veille sur les solutions cloud et vector DB
S3	Conception de l'architecture
S4	Backlog et estimation des coûts
S5–S6	Développement : mémoire conversationnelle + géolocalisation
S7	Développement : recherche web complémentaire
S8	Développement : monitoring
S9	Tests et validation
S10	Finalisation et présentation

Livrables :

- Étude de design du MVP (le présent rapport)
- Architecture technique documentée
- Backlog priorisé (méthode MoSCoW)
- Estimation des coûts (build & OPEX)
- Support de présentation

Macro backlog des fonctionnalités

Le backlog est priorisé selon la méthode MoSCoW (Must-Have / Nice-to-Have). Les estimations sont exprimées en jours de développement.

ID	Fonctionnalité	Priorité	Estimation
F01	Socle RAG (consolidation du POC)	Must-Have	2 j
F02	Mémoire conversationnelle	Must-Have	4 j
F03	Localisation / filtres géographiques	Must-Have	4 j
F04	Recherche web complémentaire	Must-Have	5 j
F05	Affichage des sources	Must-Have	2 j
F06	Monitoring et suivi qualité	Must-Have	3 j

F07	Infrastructure cloud scalable	Must-Have	5 j
F08	Tests et validation (dont ré-évaluation RAGAS)	Must-Have	3 j
	Sous-total Must-Have		28 j
F09	Recherche hybride (FAISS + BM25)	Nice-to-Have	4 j
F10	Reranking des résultats	Nice-to-Have	3 j
F11	Filtres avancés	Nice-to-Have	3 j
F12	Personnalisation utilisateur	Nice-to-Have	5 j
	Sous-total Nice-to-Have		15 j
	Total projet complet		43 j

Risque principal et mitigation : l'objectif du MVP est de faire remonter le Context Recall au-dessus de 0,5 grâce au filtrage géographique (F03). Si cela ne suffit pas après mise en production, les fonctionnalités F09 et F10 (recherche hybride, reranking), déjà budgétées en Nice-to-Have, prendront le relais sans remettre en cause l'architecture globale.

Architecture technique détaillée

Schéma d'architecture

Le flux général de l'application est le suivant : l'utilisateur pose une question, qui est traitée, mise en contexte, puis transformée en réponse.

Composant	Rôle
Utilisateur (Web / Mobile)	Pose la question
API Gateway	Point d'entrée sécurisé (clé d'accès + limite de requêtes)
FastAPI / ECS Fargate	Orchestration de la logique métier, sans gestion de serveur
Qdrant Cloud	Recherche vectorielle des événements + filtres géographiques
Redis (ElastiCache)	Mémoire conversationnelle (historique des échanges)
Mistral AI	Génération de la réponse à partir du contexte récupéré
smolagents	Recherche web complémentaire si l'information manque
OpenAgenda	Source des événements, mise à jour automatique quotidienne
CloudWatch + LangSmith	Suivi des performances, des erreurs et de la qualité des réponses

Choix du cloud provider et de la base vectorielle

AWS a été retenu pour trois raisons simples : c'est le fournisseur cloud le plus mature (documentation et support abondants), il propose des services « clé en main » comme ECS Fargate qui évite de gérer un serveur soi-même, et sa facturation à l'usage est adaptée à un MVP qui démarre petit.

FAISS, utilisé dans le POC, stocke les vecteurs directement dans l'application : pratique pour un prototype, mais peu adapté si plusieurs utilisateurs se connectent en même temps ou si la base grossit. Qdrant Cloud est un service externe déjà prêt à l'emploi qui gère cette montée en charge et propose surtout un filtrage géographique natif — ce qui répond directement à la faiblesse du Context Recall observée sur le POC.

Stratégie de déploiement, modularité et monitoring

Déploiement : l'application est empaquetée en conteneur (Docker) et déployée sur ECS Fargate. Les événements sont mis à jour automatiquement une fois par jour via une tâche planifiée qui interroge OpenAgenda.

Modularité : chaque brique (mémoire, géographie, recherche web, monitoring) est un service indépendant. Cela permet de faire évoluer ou remplacer un composant sans casser le reste du système.

Monitoring : CloudWatch surveille l'infrastructure (erreurs, temps de réponse) et LangSmith suit la qualité des réponses générées par l'IA. Avant chaque mise en production majeure, une nouvelle évaluation RAGAS est réalisée pour vérifier que la qualité des réponses s'est bien améliorée.

Sécurité et maîtrise des coûts

Une clé d'accès (API Key) et une limite de requêtes par utilisateur (rate limiting) sont mises en place au niveau de l'API Gateway, afin d'éviter les abus et de maîtriser les coûts liés aux appels à Mistral AI et à la recherche web.

Articulation entre données internes et recherche web

Le chatbot répond en priorité à partir des données internes (OpenAgenda). La recherche web n'intervient qu'en complément, lorsque l'information manque, et elle est présentée séparément à l'utilisateur afin de rester transparent sur l'origine de chaque information.

Estimation des coûts build & OPEX

Coût de développement initial (build) :

Poste	Détail	Montant
Build (socle Must-Have)	30 jours × 400 €	12 000 € HT

Coût d'exploitation (OPEX) mensuel estimé, selon la charge utilisateurs :

Charge	Coût OPEX mensuel estimé
Faible	37 €

Moyenne	225 €
Forte	1 215 €

Ces montants sont des estimations de cadrage, à affiner avec les calculateurs officiels et les paramètres réels de production. Proposition d'optimisation budgétaire : démarrer avec le socle Must-Have (28 jours) et dimensionner les services cloud selon l'usage réel observé, plutôt que de sur-dimensionner dès le départ.

Bilan

Rappel des étapes clés du POC au MVP

Aspect	POC	MVP
Recherche vectorielle	FAISS (locale)	Qdrant Cloud (managé, filtres géographiques)
Mémoire	Aucune	Redis — questions de suivi
Contexte	Aucun filtrage géographique	Qdrant + filtres géographiques
Web	Aucune recherche externe	smolagents — si information manquante
Qualité	Évaluation ponctuelle (RAGAS)	CloudWatch + LangSmith — suivi continu
Infrastructure	Application locale (Streamlit)	Architecture cloud scalable (AWS)

Le MVP ne remplace pas le POC : il consolide le socle existant et répond directement à ses limites mesurées.

Justification des choix techniques et méthodologiques

- AWS : maturité, services managés, facturation à l'usage
- Qdrant Cloud : filtrage géographique natif et scalabilité, réponse directe au faible Context Recall du POC
- Redis : mémoire à faible latence pour la conversation
- Priorisation MoSCoW : le socle Must-Have (28 j) répond aux quatre besoins prioritaires ; les évolutions plus avancées (hybride, reranking, personnalisation) restent prévues en Nice-to-Have

Défis rencontrés et solutions apportées

- Context Recall faible sur le POC (0,233) → priorisation du filtrage géographique en Must-Have, avec objectif cible > 0,5, et recherche hybride/reranking en solution de repli
- Choix du fournisseur cloud et de la base vectorielle → comparatif simplifié sur des critères accessibles (maturité, coût, filtres natifs)
- Risque de coûts incontrôlés → clé d'accès et limite de requêtes au niveau de l'API Gateway
- Fraîcheur des données → mise à jour automatique quotidienne depuis OpenAgenda

Conclusion

Le MVP proposé s'appuie directement sur les résultats et les limites du POC pour construire une architecture scalable, priorisée et budgétée. Le socle Must-Have (28 jours, 12 000 € HT) permet de répondre aux quatre besoins exprimés par l'équipe, tout en gardant une trajectoire claire pour les évolutions futures.

Annexes

Lien vers le portfolio professionnel

[À compléter : lien GitHub vers le portfolio de projets Data Engineer]

- Récapitulatif documenté des projets réalisés sur le parcours Data Engineer — stocké sur GitHub
- Description des projets, outils et technologies utilisés, résultats obtenus avec démonstration
- Compétences démontrées et valeur ajoutée
- Liens externes ou références utiles
- Autres documents pertinents (fichiers CSV, logs, ou autres données annexes)